

SAT Solver

Repozitár: <https://github.com/lsanarchist/satsolver>

Testovanie na dátovej sade zo zadania

Riešenie som otestoval na pôvodnej sade 35 CNF príkladov zo zadania príkazom `./run_original_35_timed.sh`. Test bežal na CPU AMD Ryzen 7 2700, Python `/usr/bin/python`, `repeat=1`. Výsledok: `cases=35, ok=35, failed=0, total_time_s=9.3451, avg_time_s=0.2670`. Najpomalší prípad bol `large/test_6.cnf` s časom 3.0135 s.

Sada	OK	SAT	UNSAT	Total	Max
small	10/10	7	3	0.4449 s	0.0494 s
medium	10/10	2	8	1.6768 s	0.8034 s
large	10/10	6	4	4.6764 s	3.0135 s
special	5/5	1	4	2.5472 s	2.2776 s
overall	35/35	16	19	9.3451 s	3.0135 s

Jednoduchý opis logiky

1. Program načíta súbor v DIMACS CNF formáte: počet premenných a zoznam klauzúl.
2. Klauzuly očistí: odstráni duplicity, ignoruje tautológie a hneď spracuje jednotkové klauzuly.
3. Potom beží CDCL vyhľadávanie. Solver si pamätá trail, teda poradie priradených literálov, rozhodovacie úrovne a dôvody priradení.
4. Po každom priradení spustí propagáciu. Ak z nejakej klauzuly zostane len jeden možný literál, tento literál sa nastaví automaticky.
5. Keď vznikne konflikt, solver z neho vytvorí novú naučenú klauzulu, vráti sa na vhodnú staršiu úroveň a pokračuje ďalej.
6. Ak sa podarí priradiť premenné bez konfliktu, výsledok je SAT a solver vypíše model. Ak konflikt vznikne už na úrovni 0, výsledok je UNSAT.

Čo zrýchľuje riešenie

- Watched literals: dlhšia klauzula sleduje iba dva literály, takže sa nemusí stále prechádzať celá.
- VSIDS-like výber premenných: premenné, ktoré sa často objavujú v konfliktoch, dostávajú vyššiu prioritu.
- Naučené klauzuly a nechronologický návrat: solver si zapamätá príčinu konfliktu a nevracia sa iba o jeden krok.
- Reštarty podľa Lubyho postupnosti: hľadanie sa občas spustí od koreňa, ale naučené klauzuly ostanú.
- Špeciálne rozpoznávače pred CDCL rýchlo potvrdia niektoré UNSAT prípady, napríklad pigeonhole, XOR a Mycielski grafové farbenia.